

葉陽甫 | 推薦信參考資料

提供趙義雄老師了解研究歷程與可舉例的具體事件

以下整理我在科展與系統開發中幾個重要轉折，供老師撰寫推薦信時參考。我主要負責 Math16、Prompt Engineering、Domain Scaffold 與 AST Active Healer。這段歷程對我最大的改變，是從「把系統做出來」，慢慢學會「怎麼證明自己的方法真的有效」。

一、研究是從實際系統問題開始的

一開始，自適應學習系統大量依賴雲端 AI，API 成本逐漸累積。我嘗試把部分工作移到本地小模型，但實測後發現，小模型雖然成本低，生成程式卻常出現格式、介面與資料流錯誤。問題因此從「怎麼省下雲端成本」變成「怎麼讓小模型少做一點，卻做得更可靠？」Scaffold、Domain Functions 與 AST Healer，就是從這個實際問題一步一步發展出來的。

這件事可以反映：自主探索、問題意識、工程實作能力，以及把實際困難轉成研究問題的能力。

二、東區科展沒有晉級，讓我第一次真正回頭檢查研究

校內科展獲優勝後，東區科展雖獲優等，但沒有晉級全國賽。這次結果讓我重新檢查研究本身，才發現當時同時處理太多技術，焦點不夠集中；各方法到底解決了什麼、真正帶來多少效果，也缺乏足夠證據。那次之後我才真正理解：「系統做得出來，不代表研究已經被證明。」

這件事可以反映：抗挫折、自省，以及遇到結果不如預期時，願意先檢查自己的方法。

三、最重要的轉折：從 97.41 重新驗證到 2.63%

初版實驗中，MCRI 從 41.00 提升到 94.39，再到 97.41，當時我一度認為方法已經證明有效。但後來重新檢查，加上旺宏評審的追問，我發現測試資料是否獨立、自建指標能否作為核心證據、Healer 到底真正修好了什麼，這些問題都還沒有回答清楚。於是我重新設計 960 個實驗單元，把規則開發與 720 個正式測試分開，並在正式測試前凍結 Prompt 與 Healer 規則。嚴格重驗後，342 個原始 FAIL 中真正安全救回 9 個，救援率只有 2.63%。

這件事可以反映：研究誠信、科學精神與批判思考。比起守住漂亮的數字，我更在意這個數字是不是真的站得住腳。

四、2.63% 之後，我沒有把數字硬做漂亮

重新分析失敗案例後，我發現 342 個 FAIL 中有 144 個屬於答案或核心演算法錯誤；這些問題若要修，就等於重新解題，已經超出 Healer 原本的角色。因此我沒有為了提高成功率而擴大修復範圍，反而把原則訂得更清楚：不讀正確答案、不改數學答案與核心演算法；如果無法由固定規格唯一判定修法，就不介入。這也讓我的研究問題從「能不能做出來」，進一步變成「在什麼條件下有效、為什麼有效」。

這件事可以反映：誠實面對限制、重視可靠性與邊界，也願意在證據不足時縮小自己的結論。

國立花蓮高級中學

學生參與學術科學競賽・團隊分工與個人實質貢獻度證明書

【第 25 屆旺宏科學獎 | 全領域全國決賽 20 強】

茲證明本校學生 **葉陽甫**（身分證字號：_____，國立花蓮高級中學），於高二至高三期間參與財團法人旺宏教育基金會主辦之「第 25 屆旺宏科學獎」。本研究於全台各高中職共 846 件參賽作品中入圍全領域全國決賽 20 強，其中電腦資訊類共 4 件入圍。以下依大會繳交之創意說明書（SA25-016，30 頁）及研究實作成果，說明團隊分工；貢獻比例由指導老師依實際情況填寫。

一、旺宏參賽研究基本資訊

研究編號	SA25-016（隊伍：工人智慧）	參賽類別	資訊學科（電腦資訊類）
參賽研究題目	〈基於人工智慧之自適應輔助學習架構探究〉		
主要研究內容	AST（抽象語法樹）程式修復、Tier A-D 四級修復流程、提示詞設定、PPO 多目標學習路徑推薦、Hybrid RAG 混合檢索		
競賽結果	入圍全領域全國決賽 20 強（電腦資訊類共 4 件入圍） （決賽口試已於 115 年 9 月 5 日完成，正式獲獎名次待 10 月 24 日頒獎典禮公布）		

二、研究內容與團隊分工

研究成員	說明書對應章節與主要工作內容	貢獻比例
葉陽甫 第一作者／ 主要研究實作者／ 決賽技術答辯人	【說明書第四章第一節：提示詞設定與 AST 修復機制；系統整合、基準測試與決賽答辯】 AST 修復機制：以 Python AST 實作程式語法檢查與修復流程，依問題類型分為 Tier A（語法閉合與全形標點）、Tier B（呼叫語法與 LaTeX 數學括號）、Tier C（領域 API 命名正規化）、Tier D（命名空間遮蔽清理與重載選取），並以回歸測試檢查修復前後結果。 提示詞設定：設計 Ab1 至 Ab2d 的提示詞設定，用於比較不同設定下的生成結果。 Math16 基準測試：建置包含 Math16 開發集 240 單元與保留集 720 單元，共 960 單元的測試與消融流程。 說明書撰寫與決賽答辯：負責說明書主要章節撰寫，並於決賽口試進行技術說明及回答評審問題。 程式實作：AST 修復模組由高中團隊自行開發，未使用外部大學實驗室提供的程式碼。	
蔡昕諾 協同成員	【說明書第四章第二節：強化學習路徑推薦】 負責公開數據集（ASSISTments）的格式整理與資料清洗；協助整理包含挫折感、無聊度等指標的多目標獎勵函數測試資料。	
林昕佑 協同成員	【說明書第四章第三節：混合檢索問答】 負責均一教育平台教材語料的文本預處理；協助收集與標註常見數學錯題診斷案例；整理 Hybrid RAG 關鍵字與向量檢索的比對測試資料。	
全體團隊合計	貢獻比例合計：	

指導老師查核說明：

本研究由本人擔任指導老師。依研究進行期間之實際分工與成果，葉陽甫同學主要負責 AST 修復機制與 Math16 基準測試流程的設計與實作，並負責研究說明書主要內容撰寫及決賽技術答辯；其餘成員分別協助資料整理、測試與相關模組之實驗工作。上述分工為本人於研究指導過程中所確認，特此證明。

此致
各大專校院招生委員會

指導老師簽章：_____（親筆簽名）
服務學校：國立花蓮高級中學 現任職稱：專任教師
聯絡電話：(03) 822-6108 電子信箱：_____
中華民國 115 年 10 月 _____ 日

國立花蓮高級中學

學生參與科技創新競賽・團隊分工與個人實質貢獻度證明書

【2026 第 23 屆育秀盃創意獎 高中職 AI 應用類金獎】

茲證明本校學生 葉陽甫（身分證字號：_____，國立花蓮高級中學），於在學期間參與財團法人育秀教育基金會主辦之「2026 第 23 屆育秀盃創意獎」，獲高中職 AI 應用類金獎。本證明依據正式提報之參賽企劃書（SA237853）與決賽技術簡報，說明團隊分工及葉陽甫同學實際負責內容如下：

一、獲獎專案基本資訊

競賽名稱	2026 第 23 屆育秀盃創意獎（高中職 AI 應用類）
專案企劃編號	SA237853
參賽作品名稱	〈自適應性輔助學習系統〉
獲獎成績	高中職 AI 應用類金獎
系統內容	教材與題庫資料處理、程式出題、學生作答介面、AI 輔助學習、路徑推薦與檢索問答等功能，並曾於花蓮高商課堂試用。

二、團隊分工與主要工作內容

研發成員	主要工作內容	貢獻比例
葉陽甫 主要開發者／企劃書主要撰寫者／決賽技術答辯人	【系統架構與主要功能開發、企劃書撰寫與決賽答辯】 - 系統架構：發起專案，規劃整體系統架構，整合前後端功能與資料流程。 - 教材與題庫資料：依年級、冊次、章節與知識節點規劃資料結構，建置題庫與學生學習歷程資料；負責伺服器環境配置、系統部署與課堂試用。 - 教材輸入與解析：實作教師講義上傳、內容解析與對應課程節點功能，並保留教師人工修訂。 - 程式出題功能：實作以 Python 程式產生題目變化與計算標準答案的功能，並加入程式錯誤處理機制。 - 學生作答與 AI 輔助功能：建置學生作答介面，支援手寫及圖片輸入，並設計分階段提示功能。 - 企劃書與決賽答辯：撰寫參賽企劃書主要內容，代表團隊進行決賽簡報，並負責主要技術說明與評審問答。	
蔡昕諾 協同成員	【路徑推薦模組資料整理與測試】 負責公開數據集（ASSISTments）的格式整理與資料清洗；協助整理強化學習推薦模組的訓練參數紀錄與測試結果。	
林昕佑 協同成員	【RAG 問答功能】 負責 RAG 問答功能相關工作。	
全體團隊合計	貢獻比例合計：	

指導老師查核說明：

本專案由本人擔任指導老師。依專案進行期間之實際分工與成果，葉陽甫同學主要負責系統架構設計、教材與題庫資料處理、程式出題功能、學生作答介面及系統部署，並負責企劃書主要內容撰寫與決賽技術答辯；其餘成員分別協助資料整理、路徑推薦模組及 RAG 問答相關工作。上述分工為本人於專案指導過程中所確認，特此證明。

此致
各大專校院招生委員會

指導老師簽章：_____（親筆簽名）
服務學校：國立花蓮高級中學 現任職稱：專任教師
聯絡電話：(03) 822-6108 電子信箱：_____
中華民國 115 年 10 月 _____ 日

我為什麼開始研究 AST Active Healer ?

個人技術核心 | AST Active Healer 與程式生成可靠性研究

01 研究起點 | 當雲端生成成本開始成為系統問題

出題系統每次生成程式都需呼叫雲端 AI，API 成本隨使用逐漸累積。我嘗試把部分工作移到本地小模型，但實測後成功率並不理想，且常出現重複性的格式、介面與資料流錯誤。

研究問題 怎麼讓小模型少做一點，卻做得更可靠？

我觀察到，同一數學單元的程式結構與常見錯誤會反覆出現，因此把問題拆成兩個可驗證的子問題：

Q1 哪些重複工作可以事先抽離？

Q2 哪些局部錯誤可用固定規則安全修復？

02 從問題逐步長出的研究設計

01 | Scaffolding Prompt

先把重複工作移出模型

預先提供題型規則與函數庫。

02 | AST Active Healer

只修能確定的局部錯誤

以固定規則修復局部錯誤。

03 | MCRI | 自建評估指標

Math Code Reliability Index

初期的生成品質評估框架

後續研究再重新檢討其限制。

Scaffolding 負責降低生成前的規劃負擔；AST Active Healer (以下簡稱 Active Healer / Healer) 負責處理生成後可明確判定的局部錯誤。

03 初版驗證 | 我第一次如何測試這個想法？

本研究以可重複執行的 Python 程式作為生成單位；每支程式可反覆產生同型但數值不同的變體題與標準答案。初版以 Gemini 3.0 Flash、Qwen 3.0 14B、Qwen 3.0 8B 交叉三種策略，共形成 9 組條件；產出 405 支 Python 程式、2,025 道數學題，並以自建指標 MCRI (0-100) 進行初步評估。

策略	內容	目的
Ab1 Bare	僅提供基礎任務指令	基礎對照
Ab2 Scaffolding	加入鷹架提示與 Domain Functions	降低模型規劃負擔
Ab3 Full Healer	在 Ab2 上加入 Active Healer	修復可判定的局部錯誤

三組逐步加入元件，比較 Ab1 → Ab2 → Ab3 的差異，即可分別看出鷹架與 Healer 的增量效果。

Scaffolding 如何進入生成流程



研究成果與外部佐證

從校內科展、東區科展到旺宏科學獎，研究在一次次外部檢驗與修正中持續推進。

從校內科展，到第一次外部檢驗

第 66 屆校內科展	校內說明書 ↗	第 66 屆東區科展（第七區）	東區說明書 ↗
日期	2026年3月1日	2026年4月28日	
作品名稱	自適應性輔助學習系統實現探究	基於人工智慧之自適應輔助學習架構探究	
結果	電腦與資訊學科 優勝	電腦與資訊學科 優等（未晉級全國賽）	

沒有晉級之後，重新檢查研究本身

01 研究問題過於分散

同時處理生成流程、Prompt、程式修復與品質評估，技術線太多，研究焦點不夠集中。

02 技術因果沒說清楚

方法已存在，但沒說清楚各自從什麼問題產生、解決什麼。

03 證據不足以拆解效果

無法回答：是哪個方法造成改善？何時有效？失敗時發生什麼？

進入第二十五屆旺宏科學獎決賽

[決賽成果報告書 ↗](#)**846**

件作品報名

20

件進入決賽

4

件資訊相關

依旺宏教育基金會 2026/7/2 官方公告，本屆 846 件作品報名、20 件進入決賽，資訊相關 4 件。「人工智慧」隊以《基於人工智慧之自適應輔助學習架構探究》進入決賽。

進入決賽，並不代表研究已經完成；反而讓我有機會重新面對：這些結果究竟證明了什麼？

進入決賽後，重新回答「怎麼證明它有效？」

01 重新聚焦問題

集中到 AI 生成數學程式的可靠度、錯誤型態與修復邊界。

02 補強對照實驗

固定模型、題目與條件，比較不同 Prompt / 工具條件。

03 重新檢視指標

保留 MCRI 方向性價值，加入 PASS / FAIL 與失敗分析。

04 保留失敗與邊界

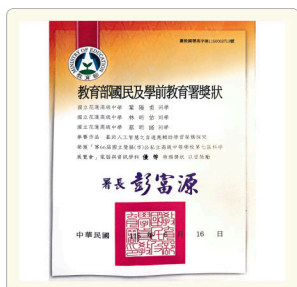
記錄 FAIL 與錯誤類型，不只展示成功案例。

以上修正於旺宏決賽準備階段進行，形成「問題 → 方法 → 實驗 → 結果 → 邊界」的決賽版研究。

正式競賽證明



校內科展優勝獎狀



東區科展優等獎狀



東區科展參賽證明



旺宏官方入圍公告 | SA25-016 | 資訊類

重新驗證：我怎麼把開發與正式測試分開？

重新限制題目來源、Prompt、Healer 規則與評估方式。

01 重新定義比較條件

決賽重新使用 Math16 題庫：16 道國中會考數學題，涵蓋 4 個數學領域，並比較 3 種模型與 4 種 Prompt 條件。

條件	提供給模型的資訊	比較重點
Ab1	僅原始題目	基礎對照
Ab2g	原始題目 + 一般程式規格	一般規格是否有幫助
Ab2d+api	原始題目 + Domain API 鷹架	領域工具帶來多少幫助
Ab2d+spec	原始題目 + 題目專屬規格鷹架	題型規格帶來多少幫助

註：Ab2d+api 與 Ab2d+spec 使用既有通用 Domain API，未針對 Math16 各題新增 API。

02 規則開發與正式測試分離

16 題 × 3 種模型 × 4 種 Prompt 條件 × 5 組固定隨機種子 = 960 個實驗單元

4 題 規則開發		12 題 正式測試
實驗單元	240	720
用途	設計 Prompt、分析錯誤、建立 Healer 規則	驗證固定後的 Prompt 與 Healer 規則
規則狀態	可依開發結果調整	全部凍結
統計用途	不納入主要成效	核心研究主要評估資料

規則開發 → 規則凍結 → 正式測試

同一題的所有結果只放在其中一組，避免同一題同時用於規則開發與正式測試。

03 正式評估怎麼判定 Healer 是否真的修好



不看答案

Healer 不讀取正確答案。

規則凍結

正式測試開始後，不再修改 Prompt 或 Healer 規則。

安全介入

只有修法可由固定規格明確判定時才修改；否則不介入。

正式測試下，Healer 真正能安全修復多少？

能修什麼、怎麼修、正式測試下救回多少

01 Healer 的能力邊界

可以修	不由 Healer 修
· 介面與輸出格式 · 局部程式結構 · 資料流與組裝問題 · 可由固定 API 規格唯一判定的局部錯誤	· 數學答案錯誤 · 核心演算法錯誤 · 題意理解錯誤 · 需要重新推理或重新解題的問題

Healer 不讀取正確答案；只有在固定規格能唯一決定修法時才介入，否則不修改。

02 正式測試下的救援判定流程



03 正式測試結果：真正救回多少？



安全救援率 = $9 \div 342 = 2.63\%$

42 個實際修改案例中，9 個轉為通過、33 個仍失敗；修改後成功比例為 21.43%。

04 為什麼只有 2.63%？看失敗類型

失敗類型	原始失敗	成功救援
程式解析 / 語法	114	1
格式 / 入口 / 輸出包裝	22	6
Domain API / 工具使用	3	0
執行 / 組裝 / 資料流	59	2
答案 / 演算法錯誤	144	0

342 個原始失敗中，144 個（約 42%）屬於答案或演算法錯誤，需要重新理解題意或改寫演算法，超出 Healer 的介入範圍；9 個成功救援主要來自介面、格式與資料流等局部問題。

重新驗證讓我看見：Healer 能可靠處理的，是修法可由固定規則唯一判定的局部錯誤；大量失敗其實發生在更前面的答案、演算法或生成規劃階段。